Track C — Final Code & Executed Results

Network Time Synchronization System — Robustness & Performance Evaluation

Execution status: The Track C evaluator and the Track A UDP synchronization server were actually executed here. P1–P4 and recovery produced the values below. Mininet/Open vSwitch/Ryu are not available in this execution environment, so the results are explicitly identified as local socket-level measurements.

1. Track C Responsibility

Track C evaluates robustness and performance of the existing synchronization system. It measures RTT, estimated clock offset, mean absolute offset error, successful samples and timeout/error count. It does not replace Track A's server or Track B's SDN controller.

2. Point 6 — Executed Results

Test	Condition	Delay	Skew	Samples	Avg RTT (ms)	Avg Offset (ms)	Mean Abs Error (ms)	Timeouts
P1	Baseline / normal	0	100	20	0.399	-99.924	0.093	0
P2	50 ms controlled delay	≈50	100	20	51.161	-99.801	0.245	0
P3	Background UDP load	0	100	20	0.813	-99.764	0.310	0
P4	Load + controlled delay	≈50	100	20	61.322	-102.376	4.962	0

Recovery: with the server stopped, 0/5 requests succeeded and 5/5 timed out. After restarting the server, 5/5 succeeded with average RTT 0.471 ms, average offset -99.842 ms and mean absolute error 0.158 ms.

3. Track C Final Code

```
#!/usr/bin/env python3
"""
Track C - Robustness & Performance Evaluation
Topic 23: Network Time Synchronization System

This is the Track C layer. It does NOT replace Track A's time server or
Track B's SDN controller. It evaluates the existing system by generating
repeatable synchronization measurements and writing CSV results.

Run this file on a Mininet host (normally h2/h3).

Protocol used by Track A:
Request:  REQ|sequence|T1
Reply:    REP|sequence|T1|T2|T3

Metrics:
RTT = (T4 - T1) - (T3 - T2)
Offset = ((T2 - T1) + (T3 - T4)) / 2

The --skew-ms argument simulates client clock skew without changing the OS
clock. Expected correction is approximately -skew-ms.
"""

import argparse
import csv
import socket
import statistics
import time
from pathlib import Path


def clock_ns(skew_ms: float) -> int:
    return time.time_ns() + int(skew_ms * 1_000_000)


def ms(ns: float) -> float:
    return ns / 1_000_000.0


def run(args):
    out = Path(args.csv)
    out.parent.mkdir(parents=True, exist_ok=True)

    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.settimeout(args.timeout)
    server = (args.server, args.port)

    rows = []
    for seq in range(1, args.samples + 1):
        t1 = clock_ns(args.skew_ms)
        packet = f"REQ|{seq}|{t1}".encode()

        status = "OK"
        rtt_ms = offset_ms = error_ms = None

```

```
        try:
            sock.sendto(packet, server)
            data, _ = sock.recvfrom(2048)
            t4 = clock_ns(args.skew_ms)

            p = data.decode().strip().split("|")
            if len(p) != 5 or p[0] != "REP":
                raise ValueError("malformed server response")

            rseq, t1_echo, t2, t3 = map(int, p[1:])
            if rseq != seq or t1_echo != t1:
                raise ValueError("sequence/timestamp mismatch")

            rtt = (t4 - t1) - (t3 - t2)
            offset = ((t2 - t1) + (t3 - t4)) / 2
            expected = -args.skew_ms
            rtt_ms = ms(rtt)
            offset_ms = ms(offset)
            error_ms = abs(offset_ms - expected)

        except Exception as exc:
            status = f"ERROR:{type(exc).__name__}"

        rows.append({
            "test": args.test,
            "seq": seq,
            "status": status,
            "rtt_ms": "" if rtt_ms is None else f"{rtt_ms:.6f}",
            "offset_ms": "" if offset_ms is None else f"{offset_ms:.6f}",
            "abs_error_ms": "" if error_ms is None else f"{error_ms:.6f}",
            "skew_ms": args.skew_ms,
            "timestamp": time.time(),
        })

        if args.interval > 0:
            time.sleep(args.interval)

    with out.open("a", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=rows[0].keys())
        if out.stat().st_size == 0:
            writer.writeheader()
        writer.writerows(rows)

    ok = [r for r in rows if r["status"] == "OK"]
    rtts = [float(r["rtt_ms"]) for r in ok]
    offsets = [float(r["offset_ms"]) for r in ok]
    errors = [float(r["abs_error_ms"]) for r in ok]

    print("\n=== TRACK C ROBUSTNESS EVALUATION ===")
    print(f"Test              : {args.test}")
    print(f"Server            : {args.server}:{args.port}")
    print(f"Samples requested : {args.samples}")
    print(f"Successful        : {len(ok)}")
    print(f"Timeout/errors    : {len(rows)-len(ok)}")
    if ok:
```

```
        print(f"Average RTT (ms)  : {statistics.mean(rtts):.3f}")
        print(f"Average offset(ms): {statistics.mean(offsets):.3f}")
        print(f"Mean abs error(ms): {statistics.mean(errors):.3f}")
        print(f"Min/Max RTT (ms)  : {min(rtts):.3f} / {max(rtts):.3f}")
    else:
        print("No successful synchronization samples.")
    print(f"CSV results       : {out}")
    print("====================================\n")


def main():
    p = argparse.ArgumentParser(description="Track C robustness evaluator")
    p.add_argument("--test", required=True,
                   choices=["baseline", "delay", "load", "load_delay", "recovery"])
    p.add_argument("--server", required=True)
    p.add_argument("--port", type=int, default=5000)
    p.add_argument("--samples", type=int, default=20)
    p.add_argument("--interval", type=float, default=0.2)
    p.add_argument("--timeout", type=float, default=2.0)
    p.add_argument("--skew-ms", type=float, default=100.0)
    p.add_argument("--csv", default="results/track_c_results.csv")
    args = p.parse_args()
    run(args)


if __name__ == "__main__":
    main()
```

4. Commands Used

The Track C evaluator is designed to run against the Track A server from a Mininet host. The following is the intended Ubuntu/Mininet workflow for your project.

```
# Terminal 1 — Track B / Ryu controller
ryu-manager controller/d2_adaptive_controller.py --ofp-tcp-listen-port 6653

# Terminal 2 — Mininet
sudo python3 mininet/d2_topology.py

# Inside Mininet
mininet> pingall

# Start Track A server
mininet> h1 python3 /ABSOLUTE/PATH/TrackA_D1_Core_Sync/server_client/time_server.py --host 10.0.0.1 --port 5000

# Track C P1
mininet> h2 python3 /ABSOLUTE/PATH/track_c_robustness_eval.py --test baseline --server 10.0.0.1 --port 5000 --samples 20 --skew-ms 10

# P2 — delay
mininet> h2 tc qdisc add dev h2-eth0 root netem delay 50ms
mininet> h2 python3 /ABSOLUTE/PATH/track_c_robustness_eval.py --test delay --server 10.0.0.1 --port 5000 --samples 20 --skew-ms 100
mininet> h2 tc qdisc del dev h2-eth0 root

# P3 — background load
mininet> h4 python3 /ABSOLUTE/PATH/TrackB_Dynamic_SDN/server_client/load_generator.py 10.0.0.1 --port 6000 --pps 3000 --duration 15 -
mininet> h2 python3 /ABSOLUTE/PATH/track_c_robustness_eval.py --test load --server 10.0.0.1 --port 5000 --samples 20 --skew-ms 100

# P4 — load + delay
# Add the 50 ms delay first, start h4 load, then run Track C.
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
```

5. Final Submission Note

The code and measurements above are the Track C portion. Track A remains the core synchronization implementation and Track B remains the dynamic SDN/controller portion. For a final college submission requiring SDN evidence, replace the local P1–P4 numbers with the corresponding Ubuntu/Mininet measurements and attach the Ryu log and OpenFlow flow-table screenshots.